- Measure and report job wait time and completion time under normal and degraded conditions

Project 23: Collaborative Editor

Implement a real-time collaborative editor (text or drawings) with multiple clients. Synchronization should be handled via a message broker or pub/sub system.

Variants:

- **A:** Text editor with live sync using WebSockets or Kafka
- **B:** Collaborative drawing canvas with update merging
- **C:** Version history and undo functionality with periodic snapshots

Validation Checklist:

- Show consistent state across multiple users
- Simulate network delay and confirm proper synchronization
- Demonstrate editing sessions and replay/undo

Project 24: Blockchain-Based Voting System

Design and implement a blockchain-based voting system where votes are submitted by clients, recorded immutably on a distributed ledger, and validated by nodes using a consensus mechanism. The system must prevent double voting, ensure transparency, and allow anonymous verification of vote inclusion.

Variants:

- **A:** A centralized blockchain ledger managed by a single node for submitting and displaying votes
- **B:** A peer-to-peer blockchain network using proof-of-authority or a simplified RAFT protocol for vote consensus
- **C:** Merkle tree integration to allow clients to verify that their vote was included without revealing their identity

Validation Checklist:

- Log all vote submissions with time, hash, and unique voter ID (or signature)
- Validate that each vote appears exactly once in the final ledger
- Demonstrate a verification method for a voter to prove their vote was counted (Merkle proof or digital receipt)

- Simulate multiple nodes accepting and verifying blocks and test the consensus process

Deliverables

Each team must deliver:

- PDF Report: follows IMRaD structure
    - **Introduction:** general theoretical overview about the topic/protocol/system and why it's relevant/useful.
    - **Methods:** describe your approach, design, and tool choices, include configurations and diagrams illustrating architecture and/or use case of your system.
    - **Results:** showcase implementation results and the system in action.
    - **Discussion:** comment on the achieved results, which challenges you faced and what could be improved/optimized.
    - **References:** links to papers or online resources used in your work.

  The report must include an architecture diagram with description of the system, performance results (charts, logs, screenshots, etc.).

- **Working system with documentation** (Code Repository with a readme file that describes how to install and use the system.)
- **Demo Video:** 1-2-minute video showing a minimal interaction/use-case of your system.

Project 25: Decentralized Chat Network: peer-to-peer messaging across multiple independent servers

Build a network of chat servers where each node can accept clients and connect to other servers.

- Messages sent on one server must propagate to others without a central coordinator.
- Implement server-to-server communication, message forwarding, and deduplication using unique message IDs.
- Handle network partitions and reconnections while preserving eventual consistency of chat history.
- Support basic features such as rooms or channels.